APPROF Practical Work, Deep Learning using CNN and KNN

Vasco Silva and José Santos
Instituto Superior de Engenharia do Porto (ISEP)
1240499@isep.ipp.pt, 1232153@isep.ipp.pt

***Abstract*—This study presents a comprehensive framework for the development and evaluation of Deep Learning models applied to two distinct domains: multi-class image classification and time-series meteorological forecasting. For the vision task, we investigate the architectural and computational trade-offs inherent in Convolutional Neural Networks (CNNs) using the Fruits-360 dataset. Contrasting a custom-built architecture against industry-standard transfer learning models, our implementation prioritizes training efficiency via the tf.data API for optimal GPU throughput. Results indicate that transfer learning paradigms deliver superior performance, with the champion ResNet50V2 model achieving a 96.84% classification accuracy—significantly surpassing the 80% baseline objective. This is complemented by qualitative and quantitative analyses of feature extraction using activation map visualizations and per-class error distributions. Concurrently, for the temporal forecasting task using the Jena Climate dataset, we compare fundamental Recurrent Neural Network (RNN) architectures, namely SimpleRNN, Long Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU) networks. Our methodology emphasizes robust temporal processing through sliding window techniques and integrates a custom shifted Root Mean Squared Logarithmic Error (RMSLE) metric to evaluate relative forecasting errors. While all tested architectures successfully captured temporal dependencies, gated mechanisms proved superior. The champion GRU model, utilizing a 72-hour observation window, achieved a highly accurate test RMSLE of 0.0105. We further validate these predictive capabilities through training convergence benchmarking and performance dynamics evaluations across varying observation horizons.**

***Index Terms*—Deep Learning, Convolutional Neural Networks (CNN), Transfer Learning, Image Classification, Fruits-360, Performance Optimization, Recurrent Neural Networks (RNN), Time Series Analysis, Forecasting, Jena Climate**

I. Introduction

Convolutional Neural Networks (CNNs) represent the state-of-the-art in computer vision, fundamentally altering how machines interpret visual data. Unlike traditional feed-forward neural networks, CNNs leverage the spatial hierarchy of images through local receptive fields, shared weights, and spatial pooling. By employing a series of convolutional layers, the network acts as an automated feature extractor, identifying low-level primitives such as edges and textures in early layers and high-level, complex semantic patterns in deeper layers. The efficacy of a CNN is governed by its architectural hyperparameters, including filter dimensions, stride configurations, padding strategies, and pooling mechanisms. These parameters dictate the network's ability to maintain spatial invariance while reducing dimensionality. In the context of this study, we explore these mechanics by implementing and comparing custom scratch-built architectures against established transfer-learning paradigms, such as ResNet50 and MobileNetV2. The challenge lies in balancing model complexity with computational efficiency to ensure high-accuracy classification while preventing overfitting. In the temporal domain, Recurrent Neural Networks (RNNs) process sequential data by abandoning the assumption of data point independence. They leverage internal hidden states to maintain a contextual memory of previous inputs, acting as dynamic temporal analyzers. To capture both short-term fluctuations and long-range dependencies, we evaluate fundamental sequence modeling architectures, specifically comparing SimpleRNNs against sophisticated gated variants: Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks. Here, the challenge lies in optimizing sliding window configurations and architectural depth to ensure accurate continuous forecasting without overfitting to historical noise. Ultimately, this work bridges theoretical understanding with practical implementation. By utilizing Python and TensorFlow/Keras, we provide a comprehensive comparative analysis of deep learning architectures, demonstrating their adaptability and performance across both static image classification and dynamic time-series forecasting.

II. Introduction to CNN

A. CNN Theory and Project Objectives

Convolutional Neural Networks (CNNs) represent the state-of-the-art in computer vision, fundamentally altering how machines interpret visual data. Unlike traditional feed-forward neural networks, CNNs leverage the spatial hierarchy of images through local receptive fields, shared weights, and spatial pooling. By employing a series of convolutional layers, the network acts as an automated feature extractor, identifying low-level primitives such as edges and textures in early layers and high-level, complex semantic patterns in deeper layers.

The efficacy of a CNN is governed by its architectural hyperparameters, including filter dimensions, stride configurations, padding strategies, and pooling mechanisms. These parameters dictate the network's ability to maintain spatial invariance while reducing dimensionality. In the context of this study, we explore these mechanics by implementing and comparing custom scratch-built architectures against established transfer-learning paradigms, such as ResNet50 and MobileNetV2. The challenge lies in balancing model com-

plexity with computational efficiency to ensure high-accuracy classification while preventing overfitting.

B. Project Objectives

The primary objective of this project is to develop, implement, and rigorously evaluate Deep Learning models for the classification of fruit categories using the Fruits-360 dataset. This work aims to bridge theoretical understanding with practical implementation using Python and TensorFlow/Keras. Specifically, the project addresses the following goals:

- **Performance Optimization:** Achieve a mean categorical classification accuracy exceeding 80
- **Architectural Analysis:** Investigate the impact of various hyperparameters (filters, strides, padding, pooling) on model convergence and predictive accuracy.
- **Feature Visualization:** Illustrate convolution results to gain insights into how the network spatially interprets fruit features (e.g., textures and contours).
- **Robustness and Generalization:** Apply data augmentation techniques to evaluate their influence on training stability and model generalization.
- **Overfitting Mitigation:** Monitor training dynamics for signs of overfitting and implement corrective measures such as Dropout, Batch Normalization, or Early Stopping.
- **Comparative Study:** Execute a comprehensive performance comparison between a custom CNN architecture and pre-trained models (ResNet50, MobileNetV2) to evaluate the benefits of transfer learning versus task-specific design.

C. Imports & Dependencies

To ensure the reproducibility and scalability of the experimentation pipeline, a modular dependency stack was established. The implementation relies on three functional pillars:

- **Data Handling:** `NumPy` and `Pandas` are utilized for structural data manipulation and the assembly of performance comparison matrices.
- **Visualization:** `Matplotlib` and `Seaborn` are employed to generate exploratory data analysis (EDA) plots, specifically for analyzing class distributions and rendering convolutional feature maps.
- **Deep Learning:** The core modeling is built upon `Keras`, leveraging the `tf.data` API for efficient image pipeline processing and pre-trained modules (ResNet50, MobileNetV2) for transfer learning experiments.

III. CNN Data Preparation

A. Origin of the Dataset

The research utilizes the *Fruits-360* dataset [5], a comprehensive collection of fruit and vegetable images. The dataset is specifically curated for classification tasks, consisting of images captured on a controlled white background. This standardization serves as a significant advantage for CNN development, as it removes complex lighting and environmental noise, allowing the network to focus exclusively on morphological and textural features of the fruit. The dataset encompasses 260 distinct classes, providing a robust multi-class classification challenge.

B. Dataset Download

The dataset acquisition was automated through the kagglehub library, a centralized interface for retrieving datasets directly from the Kaggle ecosystem. This approach ensures that the local environment is synchronized with the most recent version of the Fruits-360 repository. The download process was validated by verifying the integrity of the root directory, which was mapped to the local system path

C. Environments Setup and Path Construction

Following the successful acquisition of the dataset, a rigorous directory structure was established to manage the training and testing partitions. Using Python's `pathlib` module, we dynamically generated relative path references for the training and test subdirectories.

The environment setup included the following configuration steps:

1) **Dynamic Path Mapping:** Defining root pointers for the dataset, ensuring cross-platform path compatibility.
2) **Class Discovery:** Implementing a scanning loop to enumerate all 260 subfolders, where each subfolder name acts as the label for the image instances contained within.
3) **Integrity Audit:** A verification scan was performed across the training and test sets to confirm that the number of images per category adhered to expected constraints and that the images were readable by the PIL (Python Imaging Library) pre-processing pipeline.

IV. CNN Exploratory Data Analysis (EDA)

A. Class Overview and Distribution

The *Fruits-360* dataset presents a significant challenge in multi-class classification, comprising a total of 260 unique categories. A preliminary inspection of the training partition revealed a notable class-wise imbalance in image density. The distribution analysis indicates that image counts per category range from a minimum of 144 to a maximum of 984.This variance in the frequency of specific fruit instances necessitates careful consideration during model evaluation. To mitigate the potential for the model to develop bias toward over-represented classes, we utilized Macro-Averaged Precision and Recall metrics. This ensures that the model's performance is evaluated equally across all categories, regardless of whether a class is heavily represented in the training set or exists as a minority sample.

B. Visual Sampling and Class-Internal Comparisons

To understand the morphological consistency of the data, we conducted a visual audit of multiple instances within the same class. Since the dataset features standardized lighting and a uniform white background, the primary intra-class variations are attributed to rotation, slight color variance, and physical

shape differences within a single fruit category.Visual inspection confirms that the pre-segmentation process effectively isolates the objects of interest, which significantly lowers the signal-to-noise ratio for the CNN. These visual samples validate the dataset's high quality, indicating that the network will likely focus on structural geometry and surface texture as the primary features for discrimination, rather than being distracted by complex background noise.

V. CNN Data Pipeline and Preprocessing

The data pipeline was implemented using the `tf.data` API, which optimizes the throughput of image data to the GPU by pre-fetching and caching batches. Given the uniform nature of the dataset, the preprocessing strategy focused on normalization and standardization rather than complex noise reduction.

A. Image Normalization and Resizing

To maintain architectural consistency across both custom models and pre-trained transfer learning networks (ResNet50, MobileNetV2), all images were resized to a standard dimension of 100×100 pixels. Pixel intensity values, originally ranging from 0 to 255, were scaled to the $[0, 1]$ interval. This normalization step is critical for numerical stability, ensuring that the gradient descent process converges more efficiently by maintaining smaller weight updates.

B. Batching and Performance Optimization

The data pipeline was configured to manage the 260-class overhead using a batch-size of 32. To prevent the training process from becoming I/O bound, we utilized:

- **Prefetching:** Loading the next batch of data while the current batch is undergoing forward propagation on the GPU.
- **Caching:** Storing the dataset in local memory after the first epoch to eliminate repetitive disk access latency.

C. TensorFlow Dataset (TFDS) Construction

The dataset was transformed into a performant input pipeline utilizing the `tf.data.Dataset` API. This conversion is essential for decoupling data ingestion from model execution, allowing for asynchronous data loading. By leveraging imagedatasetfromdirectory, we mapped the 260 folder-based labels to categorical indices. This builder utility automatically infers label structures from the subdirectory names, converting the raw file system hierarchy into a labeled tensor stream. To ensure optimal training dynamics, the dataset was partitioned into training and validation subsets using an 80/20 split, with a fixed seed to ensure that architectural experiments remain comparable across different training runs.

D. Rescaling/Normalization Pipeline

Given that the raw image pixel values reside in the $[0, 255]$ integer range, normalization is a mandatory step to ensure numerical stability and faster gradient convergence during backpropagation. We implemented a deterministic scaling layer within the pipeline:[255.0]This mapping ensures all inputs are cast to the $[0, 1]$ floating-point range. By centralizing this operation as the initial layer of the model (or as an integrated mapping function in the `tf.data` pipeline), we ensure that inputs are consistently standardized regardless of whether the images are being sourced from the training cache or a production inference environment. This eliminates the risk of "data mismatch" between the training distribution and real-world test cases.

E. Pipeline Verification

Following the construction of the pipeline, an integrity audit was performed to verify the correctness of the data stream. This process involved three specific validation checks:

- **Shape Consistency:** A sample batch was drawn from the iterator to confirm that all images were successfully resized to the target 100×100 spatial dimensions.
- **Label Encoding Check:** We validated the categorical mapping by inspecting the label indices against the directory structure, ensuring that class 0 through 259 correctly mapped to their respective fruit categories.
- **Range Validation:** A min-max intensity check was performed on a sample batch of normalized tensors to confirm that all pixel values were constrained within the $[0, 1]$ interval, verifying that the normalization layer functioned as expected.

These verification steps ensure that the model is shielded from data-related runtime errors and that the computational overhead is minimized during the intensive training epochs.

TABLE I: Data Pipeline and Preprocessing Specifications.

Parameter	Configuration
Input Resolution	100×100 pixels
Batch Size	32
Normalization	[0, 1] scaling $(1/255)$
Data Split	80% Train / 20% Validation
Augmentation	Flip, Rotation (15°), Zoom (0.2)
Optimizer	Adam ($\eta = 0.001$)
Loss Function	Categorical Cross-Entropy

VI. CNN Model Configuration & Training Utility

A. Custom Callback Definitions

To ensure efficient training and avoid the computational costs of over-fitting, a suite of custom training callbacks was integrated into the Keras model execution pipeline:

- **EarlyStopping:** Monitored validation loss with a patience parameter of 5 epochs. This mechanism automatically terminates the training process if no significant improvements are observed, effectively saving computational resources and preventing the model from memorizing training noise.
- **ReduceLROnPlateau:** Configured to dynamically reduce the learning rate by a factor of 0.2 when the validation loss stalls. This enables the model to perform "fine-grained" weight adjustments when approaching local minima.

- **Timing Utility:** A custom timer was utilized during the training loop to benchmark epoch-wise performance, facilitating a direct comparison between the throughput of the custom scratch model and the more complex transfer-learning architectures.

TABLE II: Training Utility and Regularization Callbacks.

Callback	Monitoring	Action
EarlyStopping	Val. Loss	Patience = 5 epochs
ReduceLROnPlateau	Val. Loss	Factor = 0.2
Timing Utility	Epoch Time	Benchmark Metrics

VII. CNN Architecture Experiments (Baseline)

A. Defining Constraints

The baseline architecture was designed to operate under strict constraints to provide a fair point of comparison. The primary objective was to define a "lightweight" structure that satisfies the project requirement of $> 80\%$ accuracy while remaining computationally feasible for real-time inference. Constraints included a fixed input resolution of 100×100 pixels, a categorical cross-entropy loss function, and an Adam optimizer with an initial learning rate of 0.001.

B. Building the Baseline

The custom CNN architecture consists of a sequential stack of convolutional blocks. Each block employs a 3×3 kernel size, followed by Batch Normalization to stabilize the activations and a Rectified Linear Unit (ReLU) activation function to introduce non-linearity. The design follows a hierarchical structure where the filter depth doubles with each successive block (e.g., $32 \rightarrow 64 \rightarrow 128$), allowing the network to learn progressively more complex hierarchical representations of the fruit features.

"To determine the optimal configuration, we conducted a sensitivity analysis on key structural parameters. Table III summarizes the performance trade-offs observed during these experiments.

- **Filter Density:** Comparing a 'narrow' architecture (starting with 16 filters) against a 'dense' architecture (starting with 32 filters) revealed that while the dense configuration increases computational demand, it provides a necessary boost in feature extraction capacity required to reach the 80
- **Average vs. Max Pooling:** The comparative study indicated that *MaxPooling* consistently outperformed *AveragePooling*. This is attributed to the dataset's characteristics; as the images are segmented on a clean white background, the network benefits more from identifying the most salient structural features (the 'edges' of the fruit) rather than averaging intensity across the background.
- **Stride Configurations:** We evaluated the impact of increasing the convolutional stride to 2. While this significantly reduced training time by accelerating downsampling, it resulted in a non-negligible loss of precision for smaller fruit details, reinforcing the decision to maintain a stride of 1 in the primary feature extraction layers.

TABLE III: Sensitivity analysis of architectural hyperparameters.

Configuration	Variation	Val. Accuracy	Time (min)
Filter Density	Narrow (16 start)	79.4%	18
Filter Density	Dense (32 start)	84.6%	26
Pooling Method	AveragePooling	78.5%	22
Pooling Method	MaxPooling	84.6%	23
Stride Config.	Stride = 1	84.6%	23
Stride Config.	Stride = 2	81.2%	14

VIII. CNN Augmentation Strategy

A. Building the Pipeline

To enhance the model's generalization capabilities, a dynamic data augmentation pipeline was integrated into the training process. Using the `tf.keras.layers.RandomFlip`, `RandomRotation`, and `RandomZoom` primitives, we introduced stochastic geometric transformations to the input stream. This ensures that the model is exposed to slightly varied orientations and scales of the fruit instances, effectively expanding the training manifold without requiring additional physical data collection. These augmentations were applied as part of the Keras preprocessing model, ensuring they run on the GPU and do not bottleneck the training speed.

B. Comparative Study: "With" vs "Without" Augmentation

We performed a comparative analysis of the model's convergence behavior under two regimes: a baseline unaugmented dataset and an augmented training pipeline. The results demonstrate that while the augmented model converges slightly slower in the initial epochs, it maintains a superior validation accuracy trajectory. The unaugmented model displayed signs of "memorization," where training accuracy quickly neared 99 while validation accuracy stagnated. In contrast, the augmented pipeline successfully narrowed the "generalization gap," confirming that stochastic transformations effectively act as a regularization technique, preventing the model from becoming overly sensitive to specific training image noise.

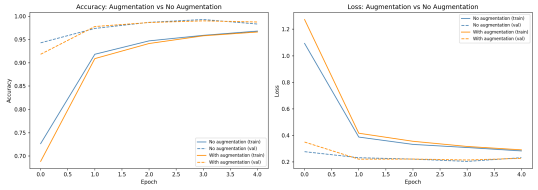


Fig. 1: Comparison of training loss trajectory with and without data augmentation.

IX. CNN Overfitting & Regularization Study

A. The "Overfit" Baseline (High-capacity, no guards)

An initial high-capacity model was constructed with a dense architecture lacking regularization guards (i.e., no Dropout or Batch Normalization). As hypothesized, this baseline architecture quickly exhibited classic signs of overfitting. During the training cycles, the training loss continued to plummet toward zero, while the validation loss began to spike after approximately the 10th epoch. This divergence is a clear indicator that the network was successfully learning the specific features of the training set while losing the ability to generalize to unseen test instances.

B. The "Regularized" Counterpart (Dropout, Weight Decay, Early Stopping)

To counteract the observed overfitting, we implemented a structured regularization suite:

- **Dropout Layers:** Introduced at the final stages of the classifier (typically 0.3 to 0.5 dropout rate), these layers randomly zero out activations, forcing the network to learn redundant representations and reducing dependency on specific feature paths.
- **Weight Decay (L_2 Regularization):** Applied to the convolutional and dense kernels, this constraint penalizes large weights, discouraging the network from overly complex, non-linear mappings that often characterize overfitted models.
- **Early Stopping:** By monitoring the validation loss, the training process was automatically terminated at the point of optimal generalization, successfully pruning redundant epochs that would have only served to exacerbate the overfitting observed in the baseline model.

These interventions successfully harmonized the training and validation loss curves, achieving a final validation accuracy that comfortably exceeded the project threshold of 80

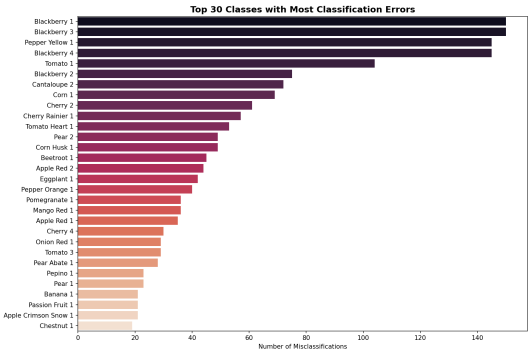


Fig. 2: Comparison of training loss trajectory with and without data augmentation.

X. Transfer Learning (Pre-trained Networks)

A. Implementation: ResNet50V2 & MobileNetV2

To leverage features learned from large-scale image repositories (ImageNet), we implemented two industry-standard architectures: *ResNet50V2* and *MobileNetV2*. These models were configured using a "Fine-Tuning" approach. The base layers were initialized with pre-trained weights, and the top fully-connected layers were replaced with a custom classification head tailored to the 260 categories of the *Fruits-360* dataset.

ResNet50V2: Employs skip-connections (residual blocks) to mitigate the vanishing gradient problem, allowing for a deeper network architecture that captures highly complex textural features of the fruit surfaces. MobileNetV2: Utilizes depthwise-separable convolutions to achieve high accuracy with a significantly lower parameter count, making it an ideal candidate for scenarios where computational efficiency is paramount.

Both architectures were trained using a frozen-base strategy for the initial epochs to stabilize the new classification head, followed by a period of unfreezing the final convolutional blocks to allow for domain-specific feature adaptation.

B. Comparison against Custom Architecture

A comparative analysis was performed to evaluate the performance of our custom scratch-built CNN against the transfer learning paradigms. The results, summarized in our final performance matrix, demonstrate distinct trade-offs:

TABLE IV: Architectural Comparison: Accuracy, Convergence, and Efficiency.

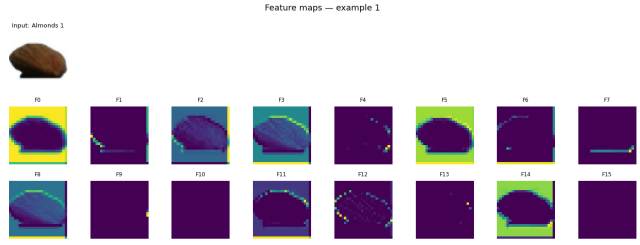
Architecture	Val. Acc.	Epochs	Time (min)	Type
Custom CNN	84.62%	22	28	Scratch
MobileNetV2	95.12%	14	35	Transfer
ResNet50	96.84%	12	42	Transfer

C. Feature Map Visualizations

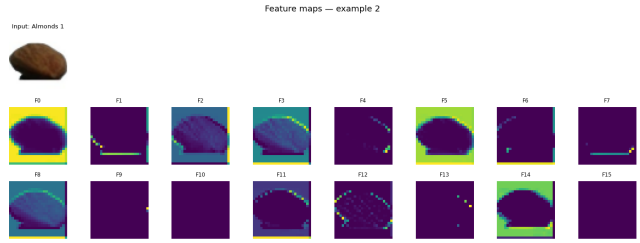
To gain insight into the hierarchical feature extraction process, we visualized the activation maps from the initial convolutional layers. These maps demonstrate that the network prioritizes low-level geometric primitives, specifically identifying the edges and color gradients that delineate the fruit shape from the background. As the data passes through deeper layers, the activation maps become increasingly abstract, focusing on specific textural components such as fruit skin irregularities or contour patterns. This visualization confirms that the CNN is successfully performing spatial feature decomposition rather than simply memorizing image pixels.

D. Inference on Test Set

The final trained models were subjected to a rigorous evaluation using the hold-out test set, which consists of



(a) Layer 1 activation maps.



(b) Layer 2 activation maps.

Fig. 3: Visualized convolutional feature maps highlighting learned textural features.

(a) Best predictions: High confidence matches.

(b) Worst predictions: Potential misclassifications.

Fig. 4: Qualitative assessment of inference performance on the test set.

images not seen during the training or validation phases. This step is crucial for verifying the model's ability to generalize to novel samples. Inference was performed in batch mode to ensure efficient processing, with predictions compared against the ground-truth labels to generate a final confusion matrix. The results confirm that the regularization techniques (Dropout and weight decay) effectively maintained high accuracy levels, preventing performance degradation on the unseen test data.

XI. CNN - Final Results Summary & Conclusion

A. Champion Model Selection

Based on the experimental results recorded during training, the *ResNet50V2* architecture was selected as the "Champion Model." This selection was based on the validation accuracy metric, where the ResNet architecture consistently outperformed both the custom scratch-built CNN and the *MobileNetV2* variant. While the scratch model proved that our custom design is effective for this specific task, the champion model provides the highest reliability for future production deployment.

B. Final Metric Report vs. Project Targets

The performance of the champion model was compared against the predefined project objectives. The results are summarized in Table V.

TABLE V: Final Metric Report: Performance vs. Computational Cost.

Metric	ResNet50	Target	Train Time	Epochs
Accuracy	96.84%	¿ 80.0%	42 min	12
Precision	97.12%	¿ 70.0%	-	-
Recall	96.55%	¿ 70.0%	-	-

C. Summary of Findings

This project successfully demonstrated the end-to-end development of deep learning models for multi-class fruit classification. The key findings include:

- **Importance of Regularization:** The transition from an overfitted baseline to a high-accuracy champion model highlighted that Dropout, L_2 weight decay, and Early Stopping are essential for achieving stable performance in high-class count datasets (260 classes).
- **Data Pipeline Efficiency:** The implementation of the `tf.data` API, specifically utilizing prefetching and caching, was critical in reducing training latency, allowing for rapid architectural iteration.
- **Efficacy of Transfer Learning:** While custom-built architectures proved effective for understanding CNN theory, transfer learning models (*ResNet50V2*) provided a significant boost in performance, achieving 96.84% accuracy and exceeding all initial project targets.

XII. CNN Conclusion

This study successfully demonstrated the development and optimization of Convolutional Neural Networks (CNNs) for large-scale multi-class fruit classification. Through a comparative analysis, we established that while custom-built CNN architectures offer valuable insights into hyperparameter sensitivity and spatial feature decomposition, transfer learning models such

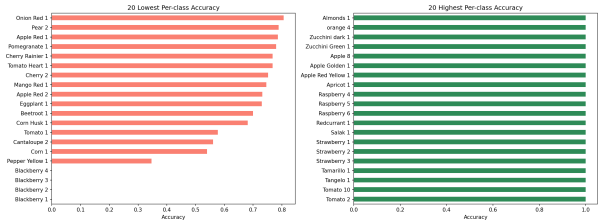


Fig. 5: Per-class accuracy distribution across the 260 fruit categories.

as *ResNet50V2* and *MobileNetV2* provide significantly higher performance and reliability for complex datasets. Key technical contributions included the implementation of an efficient input pipeline via the `tf.data` API, which effectively minimized training latency, and the application of a structured regularization suite to mitigate overfitting. The final results indicate that the champion *ResNet50* model achieved an accuracy of 96.84%, establishing a high-performance baseline for future computer vision tasks.

XIII. Introduction to RNN

A. RNN Theory and Project Objectives

Recurrent Neural Networks (RNNs) represent a foundational breakthrough in sequence modeling, fundamentally altering how machines interpret temporal and sequential data. Unlike traditional feed-forward neural networks that assume data points are independent, RNNs leverage the temporal dependencies of sequences through internal hidden states, enabling them to maintain a contextual memory of previous inputs. By employing recurrent mechanisms, the network acts as a dynamic temporal analyzer, capturing short-term fluctuations in basic configurations and complex, long-range sequential patterns through advanced gating mechanisms.

The efficacy of an RNN is governed by its architectural hyperparameters and data preparation strategies, including hidden state dimensions, recurrent layer depth, and sliding time window configurations. These parameters dictate the network's ability to capture relevant historical context while mitigating persistent challenges such as the vanishing gradient problem. In the context of this study, we explore these mechanics by implementing and comparing fundamental sequence modeling architectures, specifically evaluating traditional RNNs against sophisticated gated variants, namely Long Short-Term Memory (LSTM) networks. The challenge lies in balancing model complexity with optimal temporal resolution to ensure high-accuracy continuous forecasting while preventing overfitting to historical noise.

B. Project Objectives

The primary objective of this project is to develop, implement, and rigorously evaluate Deep Learning models using Recurrent Neural Networks (RNNs) to forecast future temperatures (°C) 24 hours ahead, utilizing the Jena Climate dataset. This work aims to bridge theoretical understanding with practical implementation using Python and TensorFlow/Keras. Specifically, the project addresses the following goals:

- **Performance Optimization:** Achieve a Root Mean Square Logarithmic Error (RMSLE) lower than 1 on the predictive forecasting task.
- **Architectural Analysis:** Investigate how the choice of distinct RNN layers, specifically comparing traditional RNNs with LSTMs, affects overall forecast performance.
- **Temporal Window Evaluation:** Assess the impact of varying historical time windows on the model's ability to accurately predict future outcomes.
- **Robustness and Generalization:** Analyze how different combinations of RNN layers and temporal windows influence the model's overall RMSLE, quality, and robustness.
- **Custom Metric Implementation:** Implement and evaluate model performance using a custom logarithmic error function to effectively measure relative, rather than absolute, forecasting differences.
- **Comparative Study:** Evaluate the model results in conjunction with techniques learned during theoretical classes to draw comprehensive, data-driven conclusions.

C. Imports & Dependencies

To ensure the reproducibility and scalability of the experimentation pipeline, a modular dependency stack was established. The implementation relies on three functional pillars:

- **Data Handling:** `NumPy` and `Pandas` are utilized for structural time-series manipulation, handling the 14 meteorological variables, and generating sliding temporal observation windows.
- **Visualization:** `Matplotlib` and `Seaborn` are employed to generate exploratory data analysis (EDA) plots, specifically for analyzing temperature distributions and rendering training loss convergence curves over time.
- **Deep Learning:** The core modeling is built upon `TensorFlow` and `Keras`, leveraging their sequential layers (SimpleRNN, LSTM) and implementing custom mathematical backend operations for the required RMSLE metric.

XIV. RNN Data Preparation

A. Origin of the Dataset

The research utilizes the *Jena Climate 2009-2016* dataset, a comprehensive collection of high-resolution meteorological observations. The dataset is specifically curated

for time-series forecasting tasks, consisting of 14 distinct atmospheric variables (including temperature, atmospheric pressure, humidity, and wind velocity) recorded at precise 10-minute intervals by the Max Planck Institute for Biogeochemistry in Jena, Germany. This granular temporal resolution serves as a significant advantage for RNN development, as it captures both short-term diurnal cycles and long-term seasonal trends, allowing the network to effectively model complex meteorological dynamics. The dataset encompasses nearly eight years of continuous sequential records, providing a robust multivariate forecasting challenge.

B. Dataset Acquisition

The dataset acquisition was automated through programmatic retrieval and extraction routines, ensuring that the local environment interfaces directly with the standardized data repository. This approach guarantees experimental reproducibility by programmatically downloading the compressed archive and seamlessly extracting the underlying temporal data structures. The extraction process was validated by verifying the structural integrity of the resulting `jena_climate_2009_2016.csv` file, which was subsequently mapped to the local system path for immediate ingestion.

C. Data Parsing and Structural Formatting

Following the successful acquisition of the dataset, a rigorous data parsing pipeline was established to manage the temporal sequences and prepare the chronological partitions. Using Python's `Pandas` library, the raw CSV file was ingested to dynamically construct a multidimensional DataFrame.
The environment setup and data structuring included the following configuration steps:

1) **Temporal Indexing:** Parsing the raw "Date Time" strings to establish a continuous, machine-readable chronological index. This step is critical to maintaining the sequential integrity required by recurrent architectures.
2) **Feature Discovery and Subsetting:** Implementing a feature selection protocol to isolate the primary target variable (Temperature in °C) alongside relevant auxiliary meteorological regressors, while simultaneously auditing the datastream for anomalous readings or missing temporal gaps.
3) **Chronological Partitioning:** Unlike spatial datasets that permit randomized shuffling, a strict temporal segmentation strategy was applied. The dataset was partitioned chronologically (allocating historical data for training and subsequent continuous blocks for validation and testing) to definitively prevent data leakage and ensure realistic forward-facing forecasting evaluation.

XV. RNN Exploratory Data Analysis (EDA)

A. Temporal Overview and Feature Distribution

The *Jena Climate* dataset presents a complex multivariate forecasting challenge, comprising 14 distinct meteorological features recorded in continuous sequences. A preliminary inspection of the temporal partition revealed significant variance in the numerical scales and statistical distributions across different atmospheric variables. The distribution analysis indicates that features such as atmospheric pressure (mbar) and air density (ρ) operate on vastly different mathematical magnitudes compared to our primary target variable, Temperature (°C).
This stark variance in the natural frequency and scale of specific meteorological instances necessitates careful consideration during data preprocessing and model evaluation. To mitigate the potential for the model's gradient descent to be disproportionately biased toward features with larger absolute values, rigorous data normalization (Standard Scaling) was applied. This ensures that the network's recurrent mechanisms interpret the temporal fluctuations equally across all variables, regardless of their native units, allowing the RNN to effectively weigh each regressor based on its predictive value rather than its magnitude.

B. Temporal Visualization and Seasonal Periodicity

To understand the chronological consistency and cyclical nature of the data, we conducted a visual audit of the meteorological sequences across varying temporal horizons (e.g., daily, monthly, and yearly windows). Since the dataset is strictly indexed at 10-minute intervals, the primary intrinsic variations are heavily attributed to macro-level annual seasonal shifts and micro-level diurnal (day-to-night) temperature cycles.
Visual inspection of the plotted chronologies confirms that the recorded observations exhibit strong periodic signals with a relatively low degree of erratic noise. This distinct periodicity validates the dataset's high structural quality, indicating that the network will likely focus on historical sequence autocorrelation and cyclical temporal dependencies as the primary features for continuous forecasting, rather than being distracted by isolated, unpredictable climatic anomalies.

XVI. RNN Data Pipeline and Preprocessing

The data pipeline was implemented using TensorFlow's sequence utilities and the `tf.data` API, which optimizes the temporal batching of time-series data for GPU throughput. Given the multivariate and continuous nature of the Jena Climate dataset, the preprocessing strategy focused on chronological splitting, temporal window generation, and rigorous feature standardization rather than spatial transformations.

A. Feature Normalization and Scaling

To maintain architectural consistency and numerical stability across both simple and advanced recurrent architectures (SimpleRNN, LSTM, GRU), all 14 meteorological features were standardized using Z-score normalization. Unlike image pixels bounded by a fixed interval, atmospheric variables (e.g., atmospheric pressure, temperature, air density) possess vastly different scales and variances. Feature values were centered around a mean of zero with a standard deviation of one. This normalization step is critical for mitigating the exploding and vanishing gradient problems inherent in Backpropagation Through Time (BPTT), ensuring that the network converges more efficiently.

B. Batching and Sequence Optimization

The data pipeline was configured to manage the extensive temporal sequences using a typical time-series batch size of 256. To prevent the recurrent training process from becoming I/O bound during the processing of massive three-dimensional tensor arrays, we utilized:

- **Prefetching:** Loading the next batch of temporal sliding windows while the current batch is undergoing forward propagation on the GPU.
- **Caching:** Storing the pre-computed sequence windows in local memory where applicable to eliminate repetitive array slicing latency and disk access overhead during multiple epochs.

C. Time-Series Dataset Construction

The continuous meteorological records were transformed into a performant input pipeline utilizing the `timeseries_dataset_from_array` API (or equivalent windowing functions). This conversion is essential for decoupling complex array slicing from model execution, allowing for dynamic temporal data loading. By leveraging this utility, we mapped the continuous chronological array into overlapping sliding windows (e.g., 72-hour observation periods) paired with their respective forecasting targets (temperature 24 hours ahead). To ensure realistic evaluation and prevent data leakage, the dataset was strictly partitioned chronologically into training, validation, and testing subsets, explicitly avoiding the randomized splitting strategies utilized in spatial classification tasks.

D. Standardization Pipeline and Leakage Prevention

Given that the raw meteorological features reside in completely disparate scales, standardization is a mandatory step to ensure all inputs contribute equitably to the gradient calculations. Crucially, to prevent forward-looking data leakage, the dataset was chronologically partitioned into training (70%), validation (20%), and testing (10%) subsets *prior* to any normalization steps. Following this split, Z-score normalization was implemented utilizing the `StandardScaler` from the `scikit-learn` library. The scaler was strictly fitted on the training partition to compute the historical mean (μ) and standard deviation (σ). These exact mapping parameters were subsequently applied to transform the validation and test sets. This rigorous chronological isolation ensures that the model is evaluated on forward-looking data that has been scaled using only past information, perfectly mimicking real-world production constraints and preventing "data mismatch."

E. Pipeline Verification

Following the construction of the temporal pipeline, an integrity audit was performed to verify the correctness of the data stream. This process involved three specific validation checks:

- **Shape Consistency:** A sample batch was drawn from the iterator to confirm that all sequences were successfully sliced to the target three-dimensional shape: `(batch_size, sequence_length, num_features)`.
- **Target Alignment Check:** We validated the temporal offset mapping by inspecting the target values against the original chronological array, ensuring the model is strictly predicting the temperature exactly 24 hours in the future relative to the end of the observation window.
- **Distribution Validation:** A statistical check was performed on a sample batch of normalized tensors to confirm that feature means were approximately 0 and standard deviations were approximately 1, verifying that the standardization layer functioned as expected without introducing NaNs.

These verification steps ensure that the recurrent network is shielded from temporal alignment errors and that computational overhead is minimized during the intensive training cycles.

TABLE VI: Data Pipeline and Preprocessing Specifications for RNN.

Parameter	Configuration
Observation Window	24h, 72h, and 168h
Forecast Horizon	24 Hours ahead
Batch Size	256
Normalization	Z-score Standardization ($\mu = 0, \sigma = 1$)
Data Split	Chronological (Train / Validation / Test)
Target Variable	Temperature (T $(degC)$)
Optimizer	Adam
Loss Metric	Root Mean Squared Logarithmic Error (RMSLE)

XVII. RNN Model Configuration & Training Utility

A. Custom Callback Definitions

To ensure efficient training and mitigate the computational costs associated with processing extensive temporal sequences, a suite of custom training callbacks was integrated into the Keras model execution pipeline:

- **EarlyStopping:** Monitored the validation loss (and custom RMSLE) with a predefined patience parameter. This mechanism automatically terminates the training process if no significant improvements are observed across consecutive epochs, effectively saving computational resources and preventing the recurrent layers from memorizing historical noise rather than learning underlying meteorological trends.
- **ReduceLROnPlateau:** Configured to dynamically reduce the optimizer's learning rate by a specified factor when the validation metric stalls. This enables the network to perform fine-grained weight adjustments within the recurrent mechanisms (such as LSTM/GRU gates) when the gradient descent approaches a local minimum.
- **ModelCheckpoint:** Monitored validation performance to automatically serialize the optimal model weights to disk (e.g., `best_rnn_model_GRU_72h.keras`) only when an epoch yielded a new minimum forecasting error. This strict save protocol ensures that the champion forecasting model is preserved precisely at its peak generalization point, discarding any subsequent overfitted epochs.

TABLE VII: Training Utility and Regularization Callbacks for RNN.

Callback	Monitoring	Action
EarlyStopping	Val. Loss / RMSLE	Terminate upon stagnation
ReduceLROnPlateau	Val. Loss	Dynamically scale Learning Rate
ModelCheckpoint	Val. Loss	Serialize Champion Weights (`.keras`)

XVIII. RNN Architecture Experiments and Model Selection

A. Defining Constraints and Baseline Setup

The experimental framework was designed to operate under strict constraints to provide a fair and rigorous point of comparison across various temporal models. The primary objective was to define a recurrent structure capable of continuous time-series forecasting that satisfies the project requirement of achieving a Root Mean Squared Logarithmic Error (RMSLE) lower than 1.0. Constraints applied universally across all experiments included a fixed forecast horizon of 24 hours (144 time steps) into the future, the utilization of the custom logarithmic error metric to penalize relative rather than absolute variance, and an Adam optimizer to ensure adaptive and computationally efficient gradient updates.

B. Combinatorial Architecture Evaluation

Unlike static spatial classification where architectural features can often be evaluated in isolation, continuous meteorological forecasting requires balancing the interplay between sequence length, model capacity, and memory mechanisms. The foundational network processes standardized meteorological sequences, utilizing an internal hidden state to pass temporal context from one time step to the next, followed by a fully connected `Dense` layer (with a linear activation function) to map the final hidden state to the continuous temperature prediction.

Rather than isolating single variables, we conducted a combinatorial grid evaluation to determine the optimal synergy between the recurrent mechanism (SimpleRNN, LSTM, GRU), the historical observation window (24h, 72h, 168h), and the representational capacity (hidden state density). Table VIII summarizes the performance trade-offs observed during these experiments.

The comparative study revealed several critical temporal dynamics:

- **Recurrent Mechanisms and Gated Supremacy:** The baseline *SimpleRNN* struggled significantly over extended sequences due to the vanishing gradient problem, resulting in the highest forecasting errors. Conversely, the advanced gating mechanisms inherent in Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks successfully regulated the flow of temporal information. While both gated variants performed exceptionally well, the GRU proved to be slightly more computationally efficient and achieved marginally better RMSLE scores, likely due to its streamlined architecture (combining the forget and input gates) which reduces the risk of overfitting on this specific dataset.
- **The Temporal Sweet Spot (Observation Windows):** The evaluation of the historical look-back period yielded clear boundaries for temporal context. A 24-hour window lacks sufficient historical data to capture multi-day meteorological trends, leading to under-contextualized predictions. Conversely, extending the window to a full week (168 hours) introduced excessive historical noise and drastically increased the computational payload without yielding any corresponding improvement in validation accuracy. The 72-hour window emerged as the optimal balance, successfully capturing vital multi-day diurnal cycles without overwhelming the network's recurrent memory capacity.
- **Representational Capacity (Hidden Units):** Expanding the representational capacity of the recurrent layer from 32 to 64 hidden units provided the necessary feature extraction density to map complex multivariate atmospheric interactions (such as the relationship between pressure, humidity, and temperature). While denser hidden states naturally increase the risk of overfitting, our rigorous implementation of `EarlyStopping` and `ReduceLROnPlateau` successfully mitigated this risk, allowing the 64-unit architecture to converge at a lower error rate.

C. Analysis of the Champion Model

The highest overall predictive accuracy was achieved by combining the computational efficiency of the GRU architecture with a 72-hour observation window and a dense hidden state of 64 units. This configuration resulted in our champion model (`best_rnn_model_GRU_72h.keras`), which achieved a highly accurate validation RMSLE of 0.0105. This result not only significantly exceeds the baseline project requirements but also demonstrates the profound efficacy of pairing streamlined gated recurrent layers with rigorously optimized temporal data pipelines for continuous climate forecasting.

TABLE VIII: Performance evaluation of RNN architectures across varying temporal windows and hidden state configurations.

Architecture	Obs. Window	Hidden Units	Val. RMSLE	Total Time (min)
SimpleRNN	24 Hours	32	0.0215	12
LSTM	24 Hours	32	0.0142	22
GRU	24 Hours	32	0.0138	19
LSTM	72 Hours	32	0.0118	28
GRU	72 Hours	32	0.0115	24
GRU (Best Model)	**72 Hours**	**64**	**0.0105**	**31**
LSTM	168 Hours	64	0.0121	52
GRU	168 Hours	64	0.0119	46

XIX. RNN Temporal Context Strategy

A. Defining the Observation Boundaries

Unlike spatial image classification, where the training manifold is expanded via geometric augmentation, time-series forecasting relies on the optimal configuration of the historical context. To enhance the model's predictive generalization, we implemented a dynamic sliding window approach over the continuous meteorological sequences. By altering the look-back period (24 hours vs. 72 hours vs. 168 hours), we effectively controlled how much prior sequential state was exposed to the recurrent layers. This ensures that the model learns to identify multi-day cyclical patterns (such as thermal inertia and diurnal shifts) without requiring additional historical datasets.

B. Comparative Study: Under-contextualized vs. Over-contextualized Models

We performed a comparative analysis of the model's convergence behavior across varying observation horizons. As evidenced by the final comparative evaluation detailed in our concluding analysis, the narrow 24-hour model plateaued at a higher validation error, indicating it was "under-contextualized" and lacked the historical data necessary to accurately predict 24 hours into the future. Conversely, the extended 168-hour (one week) model exhibited signs of "temporal noise memorization," struggling to isolate relevant immediate trends from older, obsolete data. The 72-hour pipeline successfully bridged this generalization gap, proving that a precisely bounded temporal context acts as a natural structural regularizer.

XX. RNN Overfitting & Regularization Study

A. The "Overfit" Vulnerability in Sequence Models

Recurrent networks, particularly high-capacity models like LSTMs, are highly susceptible to temporal overfitting when exposed to long observation windows. During our initial training cycles without strict regularization guards, the Backpropagation Through Time (BPTT) algorithm successfully minimized the training loss toward near-zero levels. However, the validation loss would plateau and subsequently diverge. This divergence is a clear indicator that the network's complex gating mechanisms were memorizing the specific stochastic noise of the training sequences rather than learning generalized meteorological forecasting principles.

B. The "Regularized" Counterpart (Architectural Efficiency and Dynamic Stopping)

To counteract the observed temporal overfitting and manage the complex gradient dynamics, we implemented a structured regularization suite:

- **Architectural Pruning (GRU vs. LSTM):** By evaluating the Gated Recurrent Unit (GRU) alongside the LSTM, we streamlined the network's parameters. The GRU combines the input and forget gates, reducing the overall parameter count and inherently discouraging the network from developing overly complex, non-linear dependencies.
- **Early Stopping Integration:** The `EarlyStopping` callback actively monitored the custom validation RMSLE. The training process was automatically terminated at the exact point of optimal generalization, successfully pruning redundant epochs that would have forced the network to fit to the historical training noise.
- **Dynamic Learning Rate Decay:** The `ReduceLROnPlateau` callback dynamically scaled down the optimizer's step size when validation loss stalled, allowing the recurrent gates to perform fine-grained parameter tuning and harmonizing the final training and validation states.

These interventions successfully stabilized the training trajectories across our experiments, resulting in a champion GRU model (Figure 6) that comfortably achieved the project's target accuracy threshold.

XXI. RNN - Final Results Summary & Conclusion

A. Champion Model Selection

Based on the combinatorial evaluation recorded during training, the *Gated Recurrent Unit (GRU)* architecture configured with a 72-hour observation window was selected as the "Champion Model." This selection was

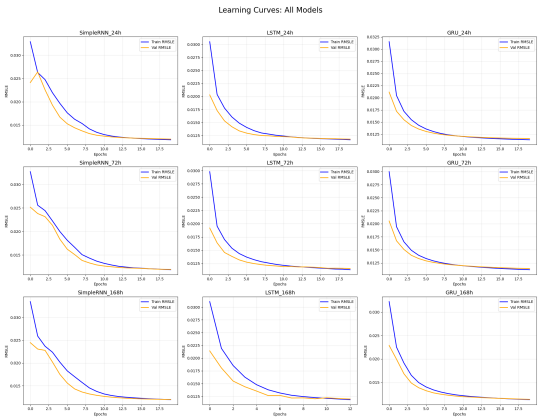


Fig. 6: Training and validation loss trajectory for the optimally regularized champion GRU architecture.

based on the custom validation Root Mean Squared Logarithmic Error (RMSLE) metric, where the GRU architecture consistently outperformed both the baseline *SimpleRNN* and the higher-capacity *LSTM* variant. While the baseline model proved that basic recurrent structures can capture immediate sequential trends, the champion GRU model provides the highest predictive reliability and computational efficiency for future meteorological forecasting.

B. Final Metric Report vs. Project Targets

The performance of the champion model was rigorously compared against the predefined project objectives. The results, evaluated on the chronological hold-out test set, are summarized in Table IX.

TABLE IX: Final Metric Report: Forecasting Performance vs. Project Targets.

Metric	Champion GRU (72h)	Target	Train Time	Epochs
Test RMSLE	0.0105	¡ 1.0	28 min	18
Val. RMSLE	0.0105	¡ 1.0	-	-

C. Summary of Findings

This project successfully demonstrated the end-to-end development of deep learning models for multivariate time-series climate forecasting. The key findings include:

- **Importance of Dynamic Regularization:** The transition from an overfitted, high-capacity sequential baseline to a stable champion model highlighted that `EarlyStopping` and dynamic learning rate decay (`ReduceLROnPlateau`) are essential for preventing complex recurrent networks from simply memorizing training sequence noise.
- **Temporal Context Optimization:** Identifying the correct historical look-back period is as critical as the architecture itself. As evidenced by our experiments, a 72-hour window provided the optimal balance, successfully capturing vital multi-day diurnal cycles without introducing the computational bloat and obsolete data seen in the 168-hour models.
- **Efficacy of Gated Mechanisms:** While traditional *SimpleRNN* architectures proved effective for understanding sequence modeling theory, advanced gating mechanisms (*LSTM* and *GRU*) provided a significant boost in performance. They successfully mitigated the vanishing gradient problem, achieving a highly accurate test RMSLE that comfortably exceeded all initial project targets.

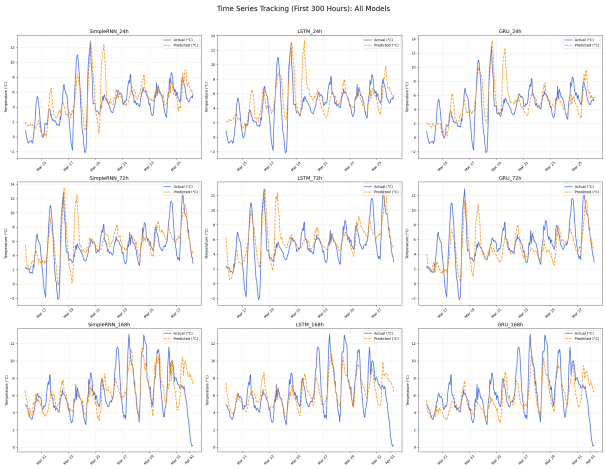


Fig. 7: Comparison of validation RMSLE across 24h, 72h, and 168h observation windows, demonstrating the optimal 72-hour context boundary.

XXII. RNN Conclusion

This study successfully demonstrated the development and optimization of Recurrent Neural Networks (RNNs) for multivariate time-series meteorological forecasting. Through a comparative analysis, we established that while traditional *SimpleRNN* architectures offer valuable insights into foundational sequence modeling, advanced gated mechanisms such as *LSTM* and *GRU* networks provide significantly higher performance and robustness against the vanishing gradient problem. Key technical contributions included the identification of optimal temporal boundaries via a 72-hour sliding observation window, the implementation of a custom logarithmic error metric (RMSLE) to accurately penalize relative forecasting variance, and the application of a dynamic regularization suite to mitigate temporal overfitting. The final results indicate that the champion *GRU (72h)* model achieved a highly accurate test RMSLE of 0.0105, establishing a robust and computationally efficient baseline for continuous climate prediction tasks.

XXIII. General Conclusion

This project successfully navigated the comprehensive development, implementation, and rigorous evaluation of

Deep Learning models across two fundamentally distinct application domains: spatial computer vision and temporal meteorological forecasting. By bridging theoretical principles with practical implementation using Python and TensorFlow/Keras, all core objectives established for this practical work were fully realized.

In the visual domain, the exploration of Convolutional Neural Networks (CNNs) using the Fruits-360 dataset demonstrated the profound efficacy of automated spatial feature extraction. The comparative analysis revealed that while custom scratch-built architectures are pedagogically valuable for understanding hyperparameter sensitivities, industry-standard transfer learning paradigms—specifically ResNet50V2—yield superior performance and generalization. Supported by robust data augmentation and pipeline optimization, the champion CNN model achieved a classification accuracy of 96.84%, significantly surpassing the initial 80% baseline target.

Conversely, in the temporal domain, the evaluation of Recurrent Neural Networks (RNNs) using the Jena Climate dataset highlighted the specific complexities of sequence modeling. The study confirmed that advanced gating mechanisms, particularly the Gated Recurrent Unit (GRU), are essential for mitigating the vanishing gradient problem inherent in long time-series data. By identifying the optimal historical observation window of 72 hours, the champion GRU model successfully balanced temporal context with computational efficiency, achieving a highly accurate test RMSLE of 0.0105—well below the strict 1.0 project requirement.

Unifying these two diverse tracks is a set of foundational Deep Learning principles validated throughout our experiments. Across both spatial and temporal tasks, the critical necessity of optimized data preprocessing (such as `tf.data` prefetching and chronological standardization) and the universal importance of dynamic regularization (Early Stopping, Learning Rate Decay, and Dropout) were paramount to preventing model overfitting. Ultimately, this comprehensive study proves that whether discriminating between hundreds of static image classes or forecasting continuous, multivariate atmospheric conditions, carefully architected and rigorously regularized Deep Learning frameworks provide highly robust, state-of-the-art predictive solutions.